p2

April 6, 2026

[1]:
```
%pip install scikit-learn pandas numpy xgboost lightgbm catboost scipy
```

```
Requirement already satisfied: scikit-learn in ./.venv/lib/python3.14/site-
packages (1.8.0)
Requirement already satisfied: pandas in ./.venv/lib/python3.14/site-packages
(3.0.1)
Requirement already satisfied: numpy in ./.venv/lib/python3.14/site-packages
(2.4.3)
Requirement already satisfied: xgboost in ./.venv/lib/python3.14/site-packages
(3.2.0)
Requirement already satisfied: lightgbm in ./.venv/lib/python3.14/site-packages
(4.6.0)
Requirement already satisfied: catboost in ./.venv/lib/python3.14/site-packages
(1.2.10)
Requirement already satisfied: scipy in ./.venv/lib/python3.14/site-packages
(1.17.1)
Requirement already satisfied: joblib>=1.3.0 in ./.venv/lib/python3.14/site-
packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in
./.venv/lib/python3.14/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
./.venv/lib/python3.14/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: graphviz in ./.venv/lib/python3.14/site-packages
(from catboost) (0.21)
Requirement already satisfied: matplotlib in ./.venv/lib/python3.14/site-
packages (from catboost) (3.10.8)
Requirement already satisfied: plotly in ./.venv/lib/python3.14/site-packages
(from catboost) (6.6.0)
Requirement already satisfied: six in ./.venv/lib/python3.14/site-packages (from
catboost) (1.17.0)
Requirement already satisfied: contourpy>=1.0.1 in ./.venv/lib/python3.14/site-
packages (from matplotlib->catboost) (1.3.3)
Requirement already satisfied: cycler>=0.10 in ./.venv/lib/python3.14/site-
packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./.venv/lib/python3.14/site-
packages (from matplotlib->catboost) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in ./.venv/lib/python3.14/site-
packages (from matplotlib->catboost) (1.5.0)
```

```
Requirement already satisfied: packaging>=20.0 in ./.venv/lib/python3.14/site-
packages (from matplotlib->catboost) (26.0)
Requirement already satisfied: pillow>=8 in ./.venv/lib/python3.14/site-packages
(from matplotlib->catboost) (12.1.1)
Requirement already satisfied: pyparsing>=3 in ./.venv/lib/python3.14/site-
packages (from matplotlib->catboost) (3.3.2)
Requirement already satisfied: narwhals>=1.15.1 in ./.venv/lib/python3.14/site-
packages (from plotly->catboost) (2.18.1)

[notice] A new release of pip is
available: 26.0 -> 26.0.1
[notice] To update, run:
python -m pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

[2]:
```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

TRAIN_DATA  = pd.read_csv('train-data.csv',  index_col='id')
TRAIN_LABEL = pd.read_csv('train-label.csv', index_col='id')
TEST_DATA   = pd.read_csv('test-data.csv',   index_col='id')

if 'subscription' in TRAIN_DATA.columns:
    TRAIN_DATA = TRAIN_DATA.drop(columns=['subscription'])
```

[3]:
```
def add_features(df):
    df = df.copy()
    df['prev_success']        = (df['poutcome'] == 'SUC').astype(int)
    df['never_contacted']     = (df['previous'] == 0).astype(int)
    df['pdays_clean']         = df['pdays'].apply(lambda x: 999 if x == -1 else␣
 ↪x)
    df['prev_contacts_log']   = np.log1p(df['previous'])
    df['duration_log']        = np.log1p(df['duration'])
    df['log_balance']         = np.log1p(df['balance'].clip(lower=0))
    df['is_debt']             = (df['balance'] < 0).astype(int)
    df['log_campaign']        = np.log1p(df['campaign'])
    df['month_sin']           = np.sin(2 * np.pi * df['month'] / 12)
    df['month_cos']           = np.cos(2 * np.pi * df['month'] / 12)
    df['long_call']           = (df['duration'] > 300).astype(int)
    df['long_call_x_success'] = df['long_call'] * df['prev_success']
    df['duration_x_prev']     = df['duration_log'] * df['prev_contacts_log']
    return df

TRAIN_DATA = add_features(TRAIN_DATA)
TEST_DATA  = add_features(TEST_DATA)
```

```
print('Shape:', TRAIN_DATA.shape)  # (29839, 29)
```

```
Shape: (29839, 29)
```

[4]:
```
from sklearn.preprocessing import OrdinalEncoder
from sklearn.model_selection import StratifiedKFold

cat_cols = ['job','marital_status','education','default_loan',
            'housing_loan','personal_loan','contact_type','poutcome']
num_cols = [c for c in TRAIN_DATA.columns if c not in cat_cols]

ENCODER = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1
                         ).fit(TRAIN_DATA[cat_cols])

def preprocess(df, encoder):
    df = df.copy()
    cat_enc = pd.DataFrame(encoder.transform(df[cat_cols]),
                           columns=cat_cols, index=df.index)
    return pd.concat([cat_enc, df[num_cols].copy()], axis=1)

def target_encode_cv(X_tr, y_tr, X_te, cols, n_splits=5):
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
    X_tr_enc, X_te_enc = X_tr.copy(), X_te.copy()
    global_mean = y_tr.mean()
    for col in cols:
        oof, te_vals = np.full(len(X_tr), global_mean), np.zeros(len(X_te))
        for tri, vali in skf.split(X_tr, y_tr):
            means = y_tr.iloc[tri].groupby(X_tr[col].iloc[tri]).mean()
            oof[vali]  = X_tr[col].iloc[vali].map(means).fillna(global_mean).
 ↪values
            te_vals   += X_te[col].reset_index(drop=True).map(means).
 ↪fillna(global_mean).values / n_splits
        X_tr_enc[col+'_te'] = oof
        X_te_enc[col+'_te'] = te_vals
    return X_tr_enc, X_te_enc

X_train  = preprocess(TRAIN_DATA, ENCODER)
X_test   = preprocess(TEST_DATA,  ENCODER)
y_train  = TRAIN_LABEL['subscription'].values
y_series = pd.Series(y_train, index=X_train.index)
X_train_te, X_test_te = target_encode_cv(X_train, y_series, X_test, cat_cols)
print('Shape:', X_train_te.shape)  # (29839, 37)
```

```
Shape: (29839, 37)
```

[5]:
```
from sklearn.ensemble import HistGradientBoostingClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
```

```
from catboost import CatBoostClassifier

scale_pos = (y_train == 0).sum() / (y_train == 1).sum()
X_arr     = X_train_te.values
X_te_arr  = X_test_te.values

best_hgbm = {
    'learning_rate'    : 0.031745219196835026,
    'max_iter'         : 1662,
    'max_leaf_nodes'   : 12,
    'max_depth'        : 8,
    'min_samples_leaf' : 43,
    'l2_regularization': 8.729921546982613,
}
best_xgb = {
    'n_estimators'    : 930,
    'learning_rate'   : 0.01711298021922337,
    'max_depth'       : 8,
    'min_child_weight': 19,
    'subsample'       : 0.9144905105460359,
    'colsample_bytree': 0.9687392914246111,
    'reg_alpha'       : 0.005704849589871292,
    'reg_lambda'      : 0.10198791505594093,
    'gamma'           : 0.5340772759198125,
    'scale_pos_weight': scale_pos,
    'eval_metric'     : 'logloss',
    'n_jobs'          : -1,
}
best_lgbm = {
    'n_estimators'     : 538,
    'learning_rate'    : 0.08154366958417066,
    'max_depth'        : 9,
    'num_leaves'       : 95,
    'min_child_samples': 74,
    'subsample'        : 0.8559039880578655,
    'colsample_bytree' : 0.6510178477019677,
    'reg_alpha'        : 8.811781685059318,
    'reg_lambda'       : 0.09599175577640591,
    'class_weight'     : 'balanced',
    'n_jobs'           : -1,
    'verbose'          : -1,
}
best_cat = {
    'iterations'         : 358,
    'learning_rate'      : 0.07621195864233186,
    'depth'              : 5,
    'l2_leaf_reg'        : 10.275311980955747,
```

```
    'bagging_temperature': 0.31171107608941095,
    'random_strength'    : 2.600340105889054,
    'auto_class_weights' : 'Balanced',
    'eval_metric'        : 'Logloss',
    'verbose'            : 0,
}
print(f'scale_pos: {scale_pos:.4f}  ')
```

```
scale_pos: 7.5597
```

[6]:
```
from sklearn.metrics import balanced_accuracy_score, roc_curve

# 3 seeds only - HGBM is slow, keep runtime under 1hr
SEEDS       = [42, 7, 123]
N_SPLITS    = 10
model_names = ['HGBM', 'XGB', 'LGBM', 'CAT']

oof_preds  = {n: np.zeros(len(y_train))  for n in model_names}
test_preds = {n: np.zeros(len(X_te_arr)) for n in model_names}

for seed in SEEDS:
    print(f'\n--- Seed {seed} ---')
    skf      = StratifiedKFold(n_splits=N_SPLITS, shuffle=True,␣
 ↪random_state=seed)
    oof_seed = {n: np.zeros(len(y_train))  for n in model_names}
    te_seed  = {n: np.zeros(len(X_te_arr)) for n in model_names}

    for fold, (tri, vali) in enumerate(skf.split(X_arr, y_train)):
        X_tr, X_val = X_arr[tri], X_arr[vali]
        y_tr        = y_train[tri]

        m = HistGradientBoostingClassifier(
            **best_hgbm, class_weight='balanced',
            random_state=seed, early_stopping=False)
        m.fit(X_tr, y_tr)
        oof_seed['HGBM'][vali] = m.predict_proba(X_val)[:, 1]
        te_seed['HGBM']       += m.predict_proba(X_te_arr)[:, 1] / N_SPLITS

        m = XGBClassifier(**best_xgb, random_state=seed)
        m.fit(X_tr, y_tr)
        oof_seed['XGB'][vali]  = m.predict_proba(X_val)[:, 1]
        te_seed['XGB']        += m.predict_proba(X_te_arr)[:, 1] / N_SPLITS

        m = LGBMClassifier(**best_lgbm, random_state=seed)
        m.fit(X_tr, y_tr)
        oof_seed['LGBM'][vali] = m.predict_proba(X_val)[:, 1]
        te_seed['LGBM']       += m.predict_proba(X_te_arr)[:, 1] / N_SPLITS
```

```
        m = CatBoostClassifier(**best_cat, random_seed=seed)
        m.fit(X_tr, y_tr)
        oof_seed['CAT'][vali]  = m.predict_proba(X_val)[:, 1]
        te_seed['CAT']        += m.predict_proba(X_te_arr)[:, 1] / N_SPLITS

        print(f'  Fold {fold+1}/{N_SPLITS} done')

    for n in model_names:
        oof_preds[n]  += oof_seed[n]  / len(SEEDS)
        test_preds[n] += te_seed[n]   / len(SEEDS)

print('\nOOF BA per model:')
for n in model_names:
    fpr, tpr, ths = roc_curve(y_train, oof_preds[n])
    t  = float(ths[np.argmax(tpr - fpr)])
    ba = balanced_accuracy_score(y_train, (oof_preds[n] >= t).astype(int))
    print(f'  {n}: BA={ba:.5f}  threshold={t:.4f}')
```

```
--- Seed 42 ---
  Fold 1/10 done
  Fold 2/10 done
  Fold 3/10 done
  Fold 4/10 done
  Fold 5/10 done
  Fold 6/10 done
  Fold 7/10 done
  Fold 8/10 done
  Fold 9/10 done
  Fold 10/10 done

--- Seed 7 ---
  Fold 1/10 done
  Fold 2/10 done
  Fold 3/10 done
  Fold 4/10 done
  Fold 5/10 done
  Fold 6/10 done
  Fold 7/10 done
  Fold 8/10 done
  Fold 9/10 done
  Fold 10/10 done

--- Seed 123 ---
  Fold 1/10 done
  Fold 2/10 done
  Fold 3/10 done
```

```
  Fold 4/10 done
  Fold 5/10 done
  Fold 6/10 done
  Fold 7/10 done
  Fold 8/10 done
  Fold 9/10 done
  Fold 10/10 done

OOF BA per model:
  HGBM: BA=0.87221  threshold=0.4051
  XGB: BA=0.87317  threshold=0.3137
  LGBM: BA=0.87439  threshold=0.3906
  CAT: BA=0.87268  threshold=0.4270
```

[7]:
```python
from scipy.stats import rankdata

def rank_norm(arr):
    return rankdata(arr) / len(arr)

oof_rank  = np.column_stack([rank_norm(oof_preds[n])  for n in model_names])
test_rank = np.column_stack([rank_norm(test_preds[n]) for n in model_names])

#   Meta-learner on rank-normalised OOF (same as original)
from sklearn.model_selection import StratifiedKFold as SKF
meta_skf  = SKF(n_splits=5, shuffle=True, random_state=99)
oof_stack = np.zeros(len(y_train))

meta_params = dict(n_estimators=300, learning_rate=0.03, num_leaves=15,
                   min_child_samples=20, class_weight='balanced',
                   random_state=42, n_jobs=-1, verbose=-1)

for fold, (tri, vali) in enumerate(meta_skf.split(oof_rank, y_train)):
    meta = LGBMClassifier(**meta_params)
    meta.fit(oof_rank[tri], y_train[tri])
    oof_stack[vali] = meta.predict_proba(oof_rank[vali])[:, 1]

final_meta = LGBMClassifier(**meta_params)
final_meta.fit(oof_rank, y_train)
test_stack = final_meta.predict_proba(test_rank)[:, 1]

#   BA-weighted average (second signal)
oof_ba_scores = {}
for n in model_names:
    fpr, tpr, ths = roc_curve(y_train, oof_preds[n])
    t = float(ths[np.argmax(tpr - fpr)])
    oof_ba_scores[n] = balanced_accuracy_score(y_train, (oof_preds[n] >= t).
 ↪astype(int))
```

```python
weights = np.array([oof_ba_scores[n] for n in model_names])
weights = (weights - weights.min()) / (weights.max() - weights.min() + 1e-9)
weights /= weights.sum()
print('BA-proportional weights:')
for n, w in zip(model_names, weights): print(f'  {n}: {w:.4f}')

oof_wavg  = oof_rank  @ weights
test_wavg = test_rank @ weights

#   60/40 blend (same ratio as original)
BLEND_W    = 0.6
oof_blend  = BLEND_W * oof_stack  + (1 - BLEND_W) * oof_wavg
test_blend = BLEND_W * test_stack + (1 - BLEND_W) * test_wavg

fpr, tpr, ths = roc_curve(y_train, oof_blend)
best_t  = float(ths[np.argmax(tpr - fpr)])
best_ba = balanced_accuracy_score(y_train, (oof_blend >= best_t).astype(int))

print(f'\nBlended OOF BA : {best_ba:.5f}')
print(f'Threshold      : {best_t:.4f}')
print(f'Expected LB    : ~{best_ba + 0.008:.5f}')

print('\nThreshold scan:')
for t in np.arange(max(0.01, best_t-0.05), min(0.99, best_t+0.06), 0.005):
    preds = (oof_blend >= t).astype(int)
    ba    = balanced_accuracy_score(y_train, preds)
    mark  = ' <- best' if abs(t - best_t) < 0.003 else ''
    print(f'  {t:.3f} | {preds.sum():6d} | {ba:.5f}{mark}')
```

```
BA-proportional weights:
  HGBM: 0.0000
  XGB: 0.2659
  LGBM: 0.6041
  CAT: 0.1300

Blended OOF BA : 0.87358
Threshold      : 0.6088
Expected LB    : ~0.88158

Threshold scan:
  0.559 |   7728 | 0.86998
  0.564 |   7651 | 0.86966
  0.569 |   7572 | 0.86969
  0.574 |   7493 | 0.87038
  0.579 |   7419 | 0.87097
  0.584 |   7338 | 0.87154
  0.589 |   7262 | 0.87249
```

```
0.594 |   7194 | 0.87297
0.599 |   7135 | 0.87328
0.604 |   7062 | 0.87320
0.609 |   6999 | 0.87358 <- best
0.614 |   6932 | 0.87290
0.619 |   6861 | 0.87295
0.624 |   6803 | 0.87324
0.629 |   6751 | 0.87260
0.634 |   6699 | 0.87229
0.639 |   6656 | 0.87229
0.644 |   6614 | 0.87212
0.649 |   6578 | 0.87264
0.654 |   6538 | 0.87291
0.659 |   6497 | 0.87271
0.664 |   6455 | 0.87189
0.669 |   6402 | 0.87127
```

[8]:
```
test_classes = (test_blend >= best_t).astype(int)
print(f'Distribution - 0: {(test_classes==0).sum()}, 1: {test_classes.sum()} '
      f'(pos-rate {test_classes.mean()*100:.1f}%)')

submission = pd.DataFrame({'id': TEST_DATA.index, 'subscription': test_classes})
submission.to_csv('submission_p2.csv', index=False)
print('Saved ')
```

```
Distribution - 0: 15227, 1: 4666 (pos-rate 23.5%)
Saved
```

[ ]:

[ ]: